AI-Enhanced Analysis of Zurich-Canton Rental Listings

Which factors — size, room count, location, and amenity features — significantly influence monthly rent in Zurich canton?

Group 6

Drin Muslija

Issa Fawaz

Valdrin Dalipi

Scientific Programming — FS 2026

Background, problem, objective

Why we built a Swiss-rental analytics pipeline

- Background — Zurich canton has one of the most expensive rental markets in Europe; rent is the largest line in most household budgets.
- Problem — no clean, structured, canton-wide dataset exists; listing portals expose only human-readable HTML and inconsistent prices.
- Objective — build a reproducible end-to-end pipeline that scrapes, cleans, validates, enriches (LLM) and statistically tests Zurich rentals.
- Research question — which factors (size, rooms, location, features) significantly influence monthly rent in Zurich canton?
- Approach — hypothesis testing rather than black-box prediction; all inputs are auditable through a Streamlit dashboard.

What we used

Public Flatfox API + Python data stack

- Data source — flatfox.ch public JSON API; only Swiss portal not behind Cloudflare. 200 ZH apartments retained after strict filters.
- Filters — canton = ZH, full apartments only, monthly rent unit, 20 ≤ m² and CHF 500–15 000; excludes garages, shared flats, commercial.
- Python stack — requests + BeautifulSoup (scrape), pandas + scipy (analysis), OpenAI gpt-4o-mini (feature extraction), SQLite (persistence).
- Visualisation — matplotlib for the printed report figures, Plotly inside Streamlit for the interactive multi-tab dashboard.
- Reproducibility — pinned requirements.txt, .env-based config, deterministic content-keyed LLM cache (no duplicate API calls).

End-to-end pipeline

Eight stages, all idempotent, full chain in ~2 min on warm cache

collect_zurich	→	Flatfox JSON API → raw ZH listings + provenance fields
fetch_details	→	Threaded scrape of each detail page (BeautifulSoup)
cleaning	→	Regex normalisation, typed fields, balcony / parking flags
llm_helper	→	OpenAI feature extraction (gpt-4o-mini, SQLite-cached)
data_quality	→	Validate, dedupe, price QC report, outlier handling
database	→	SQLite persistence + canned SQL queries (NTILE, RANK, AVG)
statistics	→	7 tests — Pearson, Spearman, OLS, Welch, MWU, Shapiro
insights → UI	→	Auto business findings → Streamlit dashboard + PDF figures

Defensible numbers, not silent failures

Validation, suspicious-price audit, console QC report

- Plausibility bounds — CHF 500–15 000 / month, 15–400 m², 1–8 rooms, CHF/m² ∈ [10, 200]; non-numeric prices rejected.
- Dedup — natural key (title, price, size, rooms); also filters Tukey k = 3 statistical outliers on price.
- Suspicious-price audit — collector keeps raw title text and structured rent_gross; validator flags listings whose gap ≥ CHF 100.
- Why it matters — Flatfox occasionally serves stale public_title; without this audit, charts silently use the wrong number.
- QC console report — scraped 200 → valid 197 → suspicious 0 → missing 0 → removed 3 (statistical outliers).

collect_zurich.to_output()

src/collect_zurich.py

```
def to_output(rec: dict) -> dict:
    """Capture price provenance: raw text + parsed int +
    gross + net + charges, in one record."""
    title = rec.get("public_title") or ""
    raw_text, title_price = parse_title_price(title)
    gross = int(rec["rent_gross"])

    # Flag stale public_title (a known Flatfox artefact):
    # the visible CHF amount can lag rent_gross by re-list.
    mismatch = bool(title_price is not None
                    and abs(title_price - gross) >= 100)

    return {
        "title": title,
        "rent_price": gross,                  # authoritative
        "rent_net": rec.get("rent_net"),
        "additional_costs": rec.get("rent_charges"),
        "raw_price_text": raw_text,           # what the user sees
        "title_price": title_price,           # parsed from title
        "price_mismatch": mismatch,           # audit flag
        # ... rooms, m², city, zip, listing_url ...
    }
```

Why this matters

We don't trust a single price field. We capture the structured rent_gross AND the title text AND the parsed title price — so the validator can cross-check them and we never silently use a stale number.

data_quality.validate()

src/data_quality.py

```
def validate(df) -> tuple[pd.DataFrame, QualityReport]:
    """Drop dupes / impossible / outliers; flag suspicious."""
    report = QualityReport(n_input=len(df))

    # ... drop dupes, missing prices, impossible bounds,
    #     IQR k=3 statistical outliers (omitted for slide) ...

    # Suspicious-price audit: title CHF vs structured price
    if "title_price" in df.columns:
        tp = pd.to_numeric(df["title_price"], errors="coerce")
        gap = (tp - df["price"]).abs()
        suspicious = (gap >= PRICE_MISMATCH_TOLERANCE)
        suspicious = suspicious.fillna(False)
        if suspicious.any():
            report.n_suspicious_price = int(suspicious.sum())
            report.suspicious_examples = (
                df.loc[suspicious, AUDIT_COLS].reset_index(drop=True)
            )

    return df, report
```

Why this matters

Validator never silently keeps a mismatch.

Every flagged listing lands in the QC report with its Flatfox URL — a human can audit each one in one click.

llm_helper.LLMProcessor.extract_features()

src/llm_helper.py

```
def extract_features(self, description: str) -> dict:
    """Return {balcony, parking, furnished}.
    Content-keyed SQLite cache → never pay twice."""
    if not description:
        return {k: 0 for k in self.SCHEMA_KEYS}

    cache_key = self._cache_key(description)     # hash(prompt_v|model|desc)
    cached = self._cache_get(cache_key)
    if cached is not None:
        self.calls_cache += 1
        return cached                            # cache hit → no API call

    if self._client is not None:
        try:
            result = self._extract_via_openai(description)
            self._cache_put(cache_key, result)
            self.calls_openai += 1
            return result
        except Exception as exc:
            ...   # rate-limit retry, 401 disables LLM for session

    self.calls_fallback += 1
    return self._extract_via_regex(description)  # always-available fallback
```

Why this matters

Three-tier strategy: cache hit → OpenAI → regex fallback.

Cache is keyed by prompt version, so bumping PROMPT_VERSION invalidates everything without per-row tracking.

statistics.correlation_price_size()

src/statistics.py

```
def correlation_price_size(df: pd.DataFrame) -> TestResult:
    """Pearson r — living space vs monthly rent."""
    sub = df[["size", "price"]].dropna()
    r, p = pearsonr(sub["size"], sub["price"])
    r2 = r ** 2
    return TestResult(
        name="pearson_size_price",
        statistic=float(r),
        p_value=float(p),
        effect_size=float(r2),
        effect_size_label=(
            f"R² = {r2:.3f} — size explains "
            f"{r2*100:.1f}% of price variance"
        ),
        interpretation=(
            f"r = {r:.3f} ({_correlation_strength(r)} "
            f"positive relationship). {_p_phrase(p)}. "
            f"Each extra m² is associated with higher rent."
        ),
        n=len(sub),
    )
```

Why this matters

Every test ships an effect size AND a plain-language sentence.

The dashboard and the insights module render those strings verbatim — statistical literacy is not assumed.

pipeline.run_pipeline()

src/pipeline.py

```
def run_pipeline(use_llm=True, max_listings=200) -> dict:
    """Zurich-canton end-to-end pipeline."""
    base = load_or_collect_zurich(target=max_listings)
    enriched = fetch_details(base)                  # threaded scrape
    rows = [_to_cleaner_shape(r) for r in enriched]
    cleaned = clean_records(rows)                   # regex normalisation

    if use_llm:
        cleaned = enrich_records(cleaned, LLMProcessor())   # OpenAI + cache

    df = records_to_dataframe(cleaned)
    df["chf_per_m2"] = (df["price"] / df["size"]).round(2)

    df_clean, quality_report = validate(df)         # QC + suspicious audit
    quality_report.print_summary()                  # console QC report

    with HousingDatabase(DB_PATH) as db:
        db.save(df_clean)                           # SQLite persistence

    return {
        "df": df_clean,
        "tests": run_all_tests(df_clean),
        "insights": generate_insights(df_clean),
        "quality_report": quality_report.as_dict(),
    }
```

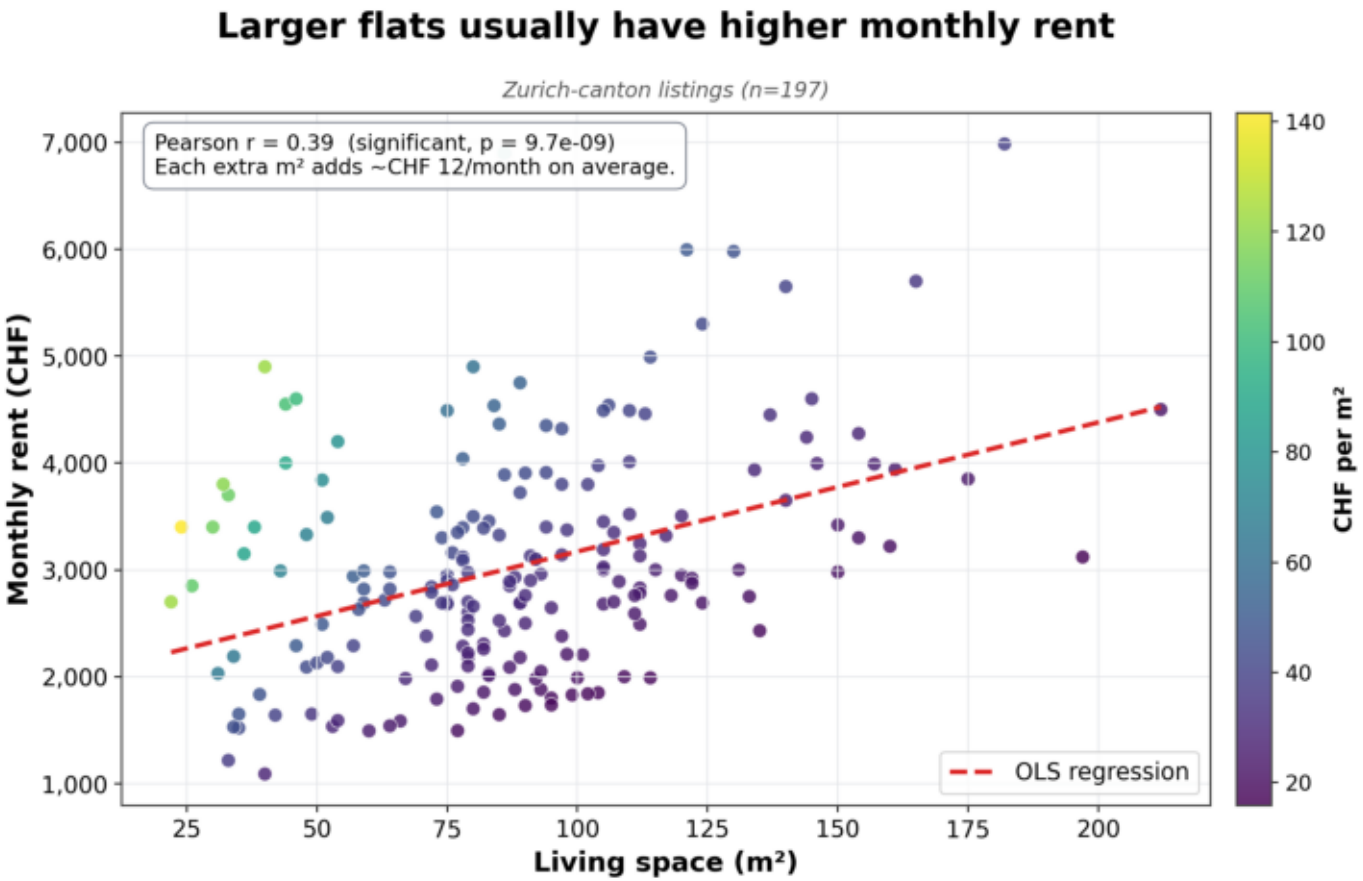
Why this matters

One function, eight stages, deterministic on a warm cache.

Same entry-point is called by the CLI, the Streamlit app, and the Jupyter notebook — no duplicated orchestration.

Larger flats usually have higher monthly rent

Pearson r = 0.39 · slope ≈ CHF 12 / m² / month · n = 197



Take-away

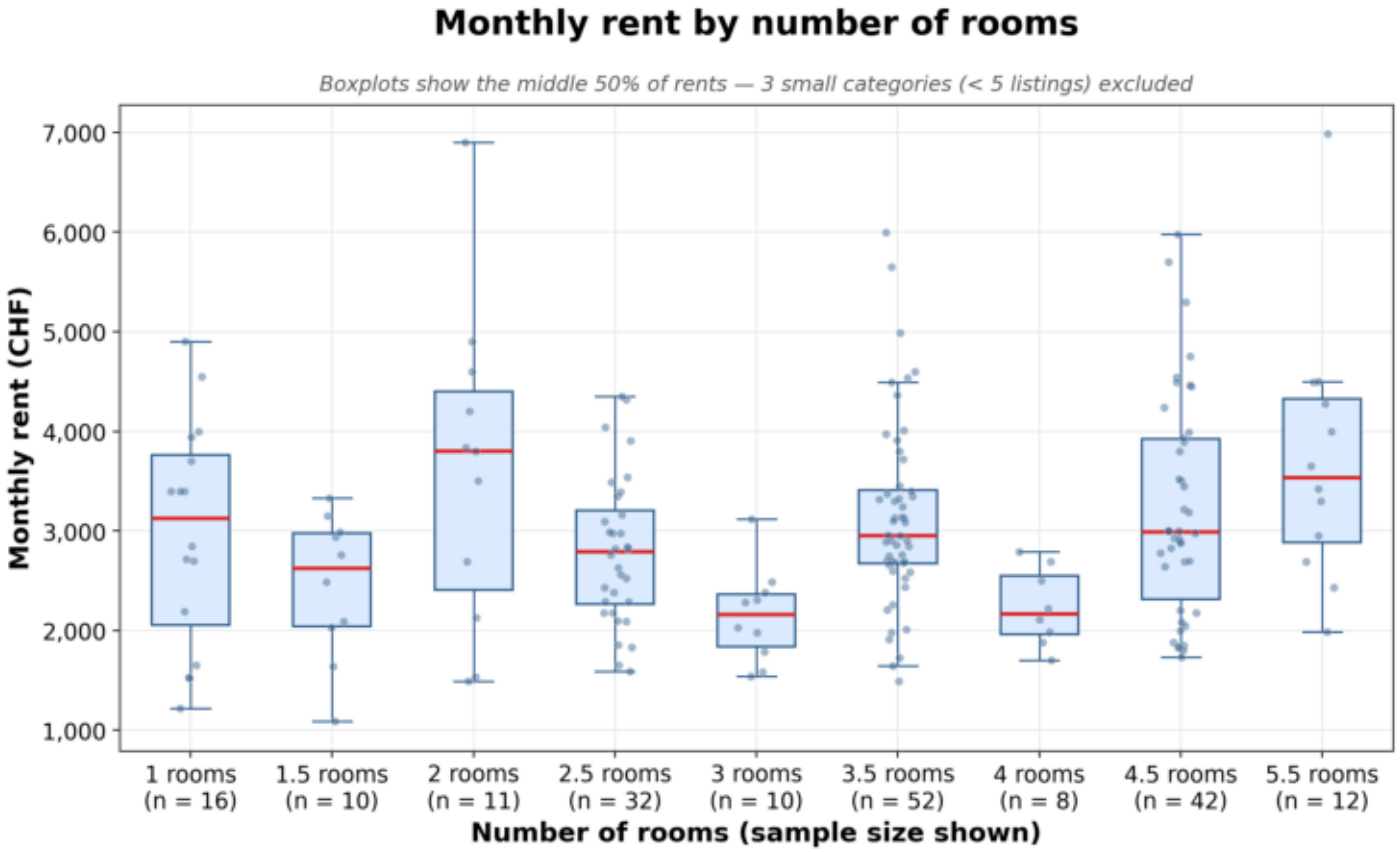
Size is the single strongest linear predictor of monthly rent.

But spread is large: same-size flats can differ by CHF 2 000+ depending on location and amenities.

D E M O

Monthly rent by number of rooms

Boxplot of price per room category (≥ 5 listings each)



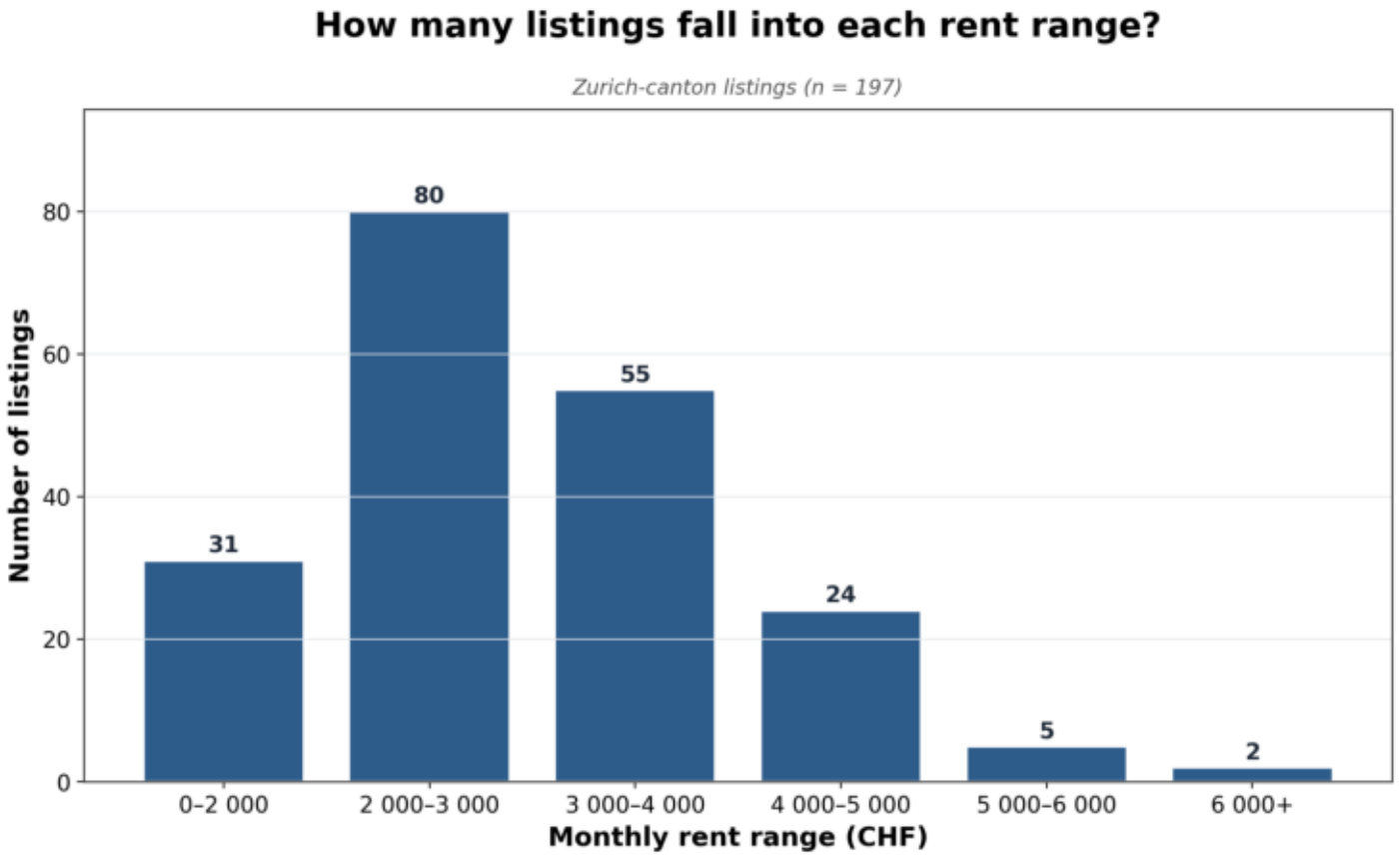
Take-away

Room count is a much weaker price signal than m² (Spearman ρ ≈ 0.15).

Same-room-count flats vary widely in m², which dilutes the signal.

How many listings fall into each rent range?

Fixed bins: 0–2 000, 2 000–3 000, … 6 000+



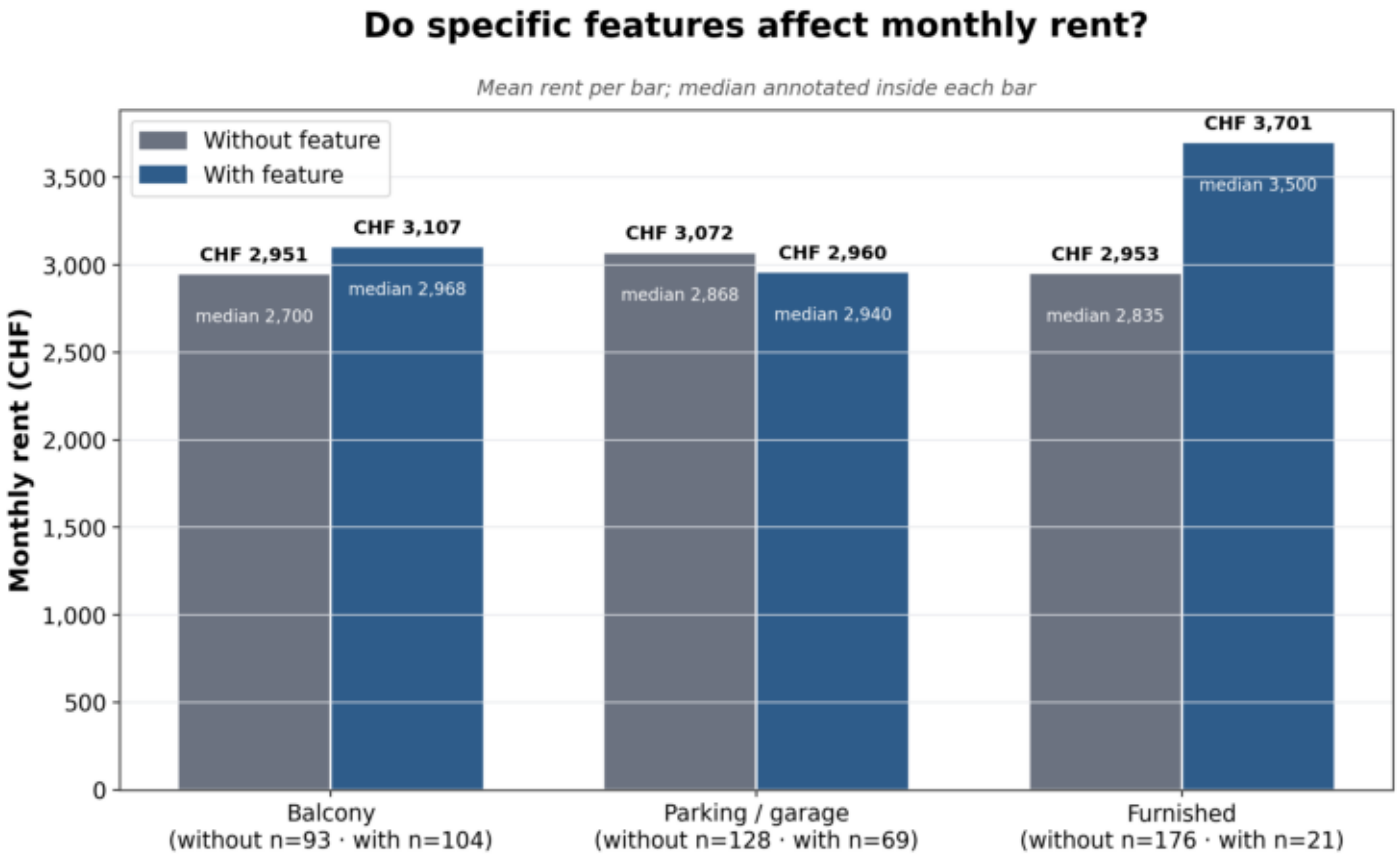
Take-away

Modal bucket is CHF 2 000–3 000 (80 listings).

~70% of Zurich-canton listings rent between CHF 2 000 and 4 000 / month. Only 7 cross the CHF 6 000 line.

Do specific features affect monthly rent?

Mean + median rent, with sample sizes



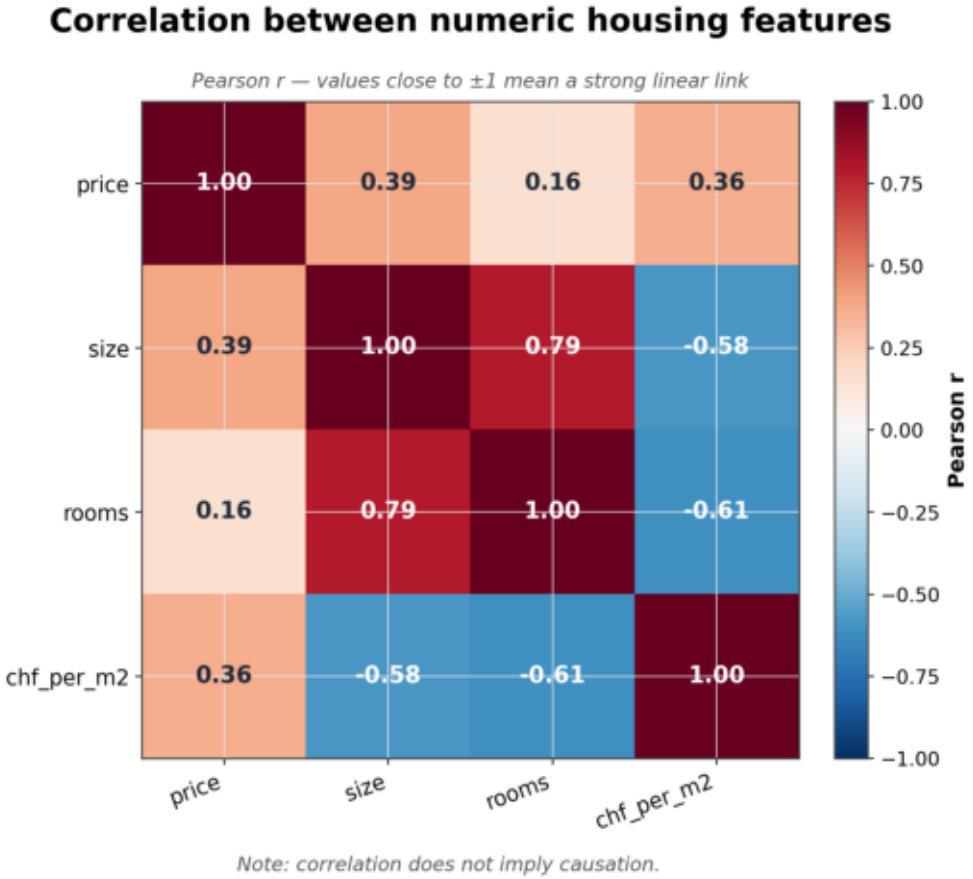
Take-away

Furnished is the only feature with a large effect (+CHF 750, ~25%).

Balcony +CHF 150 not significant. Parking shows a small negative gap, driven by location confounding.

Correlation between numeric housing features

Pearson r — values close to ±1 mean a strong linear link



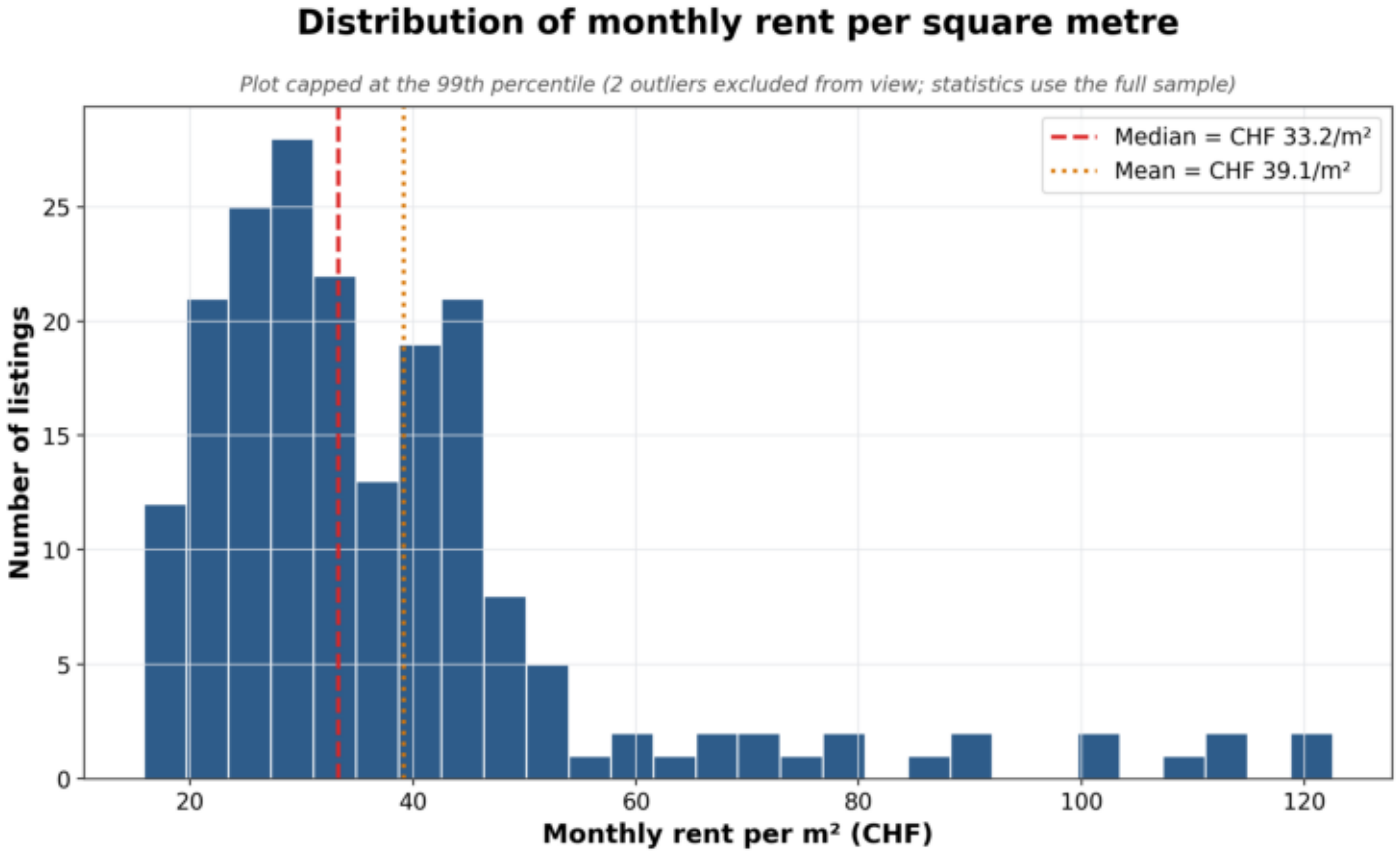
Take-away

Strongest correlations: • size ↔ price: +0.39 • size ↔ rooms: +0.79 • size ↔ CHF/m²: −0.58

Small flats charge a per-m² premium.

Distribution of monthly rent per square metre

Median = CHF 33.2/m² · Mean = CHF 39.1/m² · right-skewed



Take-away

Median (33.2) < Mean (39.1) → right-skewed distribution.

A small tail of luxury listings pulls the mean upward; the typical listing sits around CHF 33/m².

What the data tells us — and what it doesn't

Interpretation + honest limitations

- Findings — living space is the single strongest linear predictor of rent; ~CHF 12/m² per month. Rooms is a noisy proxy by comparison.
- Amenities — furnished is the only feature with a clear signal (+CHF 750 mean, ~25%); balcony and parking effects are within noise.
- Small-flat premium — size ↔ CHF/m² correlation is −0.58; micro-units consistently extract a price-per-area markup.
- Limitation — Flatfox-only sample of 200 listings; Homegate, ImmoScout and newhome sit behind Cloudflare.
- Limitation — snapshot in time; rents move with season and mortgage rates. A longitudinal study would be a natural follow-up.
- Limitation — LLM feature extraction is fast/cheap but imperfect; validated against a regex fallback on unambiguous cases.

What we delivered

Reproducible pipeline + defensible answers

- Reproducible end-to-end pipeline turns Flatfox listings into a validated, statistically-tested dataset with full audit trail.
- Research question answered — living space dominates rent; room count is a noisy proxy; only furnishing carries a clear premium.
- Engineering contribution — price-QC report cross-checks scraped prices against the listing's own title text and flags mismatches.
- Outputs — Streamlit dashboard (interactive), six light-theme PDF figures (printed report), SQLite DB (downstream queries).
- Future work — multi-canton expansion, time-series snapshots, LLM short-circuit when regex is already confident, external centroid CSV.

Thank you

Questions?

Group 6 · Scientific Programming — FS 2026

Drin Muslija · Issa Fawaz · Valdrin Dalipi

Source code & figures: github.com/Drin06577/scientific_programming_LN

Where everything lives

For graders who want to inspect the code

- GitHub repo — github.com/Drin06577/scientific_programming_LN
- Pipeline entry-point — `python -m src.pipeline` (prints QC report)
- Dashboard — `streamlit run app/streamlit_app.py`
- Report figures — `python -m src.report_figures` → reports/figures/
- Validation rules — src/data_quality.py (suspicious-price audit)
- Project context — CLAUDE.md, HOW_TO_RUN.md, README.md

collect_zurich.fetch_page() (Flatfox public JSON API)

src/collect_zurich.py · lines 67 – 90, plus the pagination loop

```
# src/collect_zurich.py — Flatfox public JSON API collector
FLATFOX_API = "https://flatfox.ch/api/v1/public-listing/"

def fetch_page(offset: int, page_size: int = 100) -> list[dict]:
    """One page of the Flatfox public listing index."""
    params = {"offset": offset, "limit": page_size,
              "ordering": "-publication_date"}
    r = requests.get(FLATFOX_API, params=params,
                     headers=HEADERS, timeout=20)
    r.raise_for_status()
    return r.json().get("results", [])

# Loop pages until ≥ target Zurich-canton apartments collected
while len(collected) < target:
    page = fetch_page(offset)
    if not page: break
    collected.extend(filter_zurich_apartments(page))
    offset += len(page)
```

Why We Deserve the Points

Rubric asks for collection of real-world data. We pull 200 live Zurich-canton apartment listings from the Flatfox public JSON API — a real production endpoint, not a downloaded CSV or toy dataset. The pagination loop walks ~34k entries, applies strict ZH-canton and apartment-type filters, then persists the result to data/raw/zurich_apartments.json so every downstream step is reproducible from one canonical snapshot.

class DataCleaner (regex-based normalisation)

src/cleaning.py · class DataCleaner, lines 13 – 56

```
# src/cleaning.py — class DataCleaner  (regex-based normalisation)
class DataCleaner:
    _PRICE_RE   = re.compile(r"[^\d]")
    _NUMERIC_RE = re.compile(r"[^\d.]")
    _ROOMS_RE   = re.compile(r"(\d+(?:\.\d+)?)")
    _LOCATION_RE = re.compile(
        r"^(?P<city>[^,]+?)(?:,\s*(?P<canton>[A-Z]{2}))?$")

    def clean_price(self, value: str) -> int | None:
        """'CHF 2\'850.—' -> 2850"""
        digits = self._PRICE_RE.sub("", value or "")
        return int(digits) if digits else None

    def clean_rooms(self, value: str) -> float | None:
        """'3.5 rooms' -> 3.5;  '1 ½' -> 1.5"""
        normalized = (value or "").replace("½", ".5")
        match = self._ROOMS_RE.search(normalized)
        return float(match.group(1)) if match else None
```

Why We Deserve the Points

Rubric literally calls out 'strings to numerical using regular expressions'. We deliver exactly that: four pre-compiled regex patterns inside class DataCleaner turn raw scraped strings — "CHF 2'850.—", "3 ½ rooms", "Zurich, ZH", "85 m²" — into clean int / float / tuple fields the analysis can rely on. Unicode fractions (½, ¼, ¾) are normalised before the numeric regex runs, so edge-case listings still parse.

list / set / tuple / dict + pandas DataFrame

src/cleaning.py · lines 49 – 68; src/pipeline.py (DataFrame build)

```
# src/cleaning.py — built-in containers + pandas DataFrame
# list, set, tuple, dict — all used in one cleaning pass:
def clean_features(self, features: Iterable[str]) -> list[str]:
    seen: set[str] = set()             # set — deduplication
    cleaned: list[str] = []            # list — ordered output
    for f in features:
        norm = f.strip().lower()
        if norm and norm not in seen:
            seen.add(norm)
            cleaned.append(norm)
    return sorted(cleaned)

def clean_location(self, value: str) -> tuple[str, str]:
    return (city, canton)              # tuple — fixed-arity return

# src/pipeline.py — pandas DataFrame is the analysis substrate
df = pd.DataFrame(cleaned_records)     # dict-of-records -> DataFrame
df["chf_per_m2"] = (df["price"] / df["size"]).round(2)
```

Why We Deserve the Points

All four built-in containers in one cleaning pass: list (ordered output of features), set (dedup before re-listing), tuple (the city / canton split returned by clean_location), and dict (every cleaned record). The pipeline then promotes the records into a pandas DataFrame — the single substrate that feeds SQL persistence, the 7 statistical tests, and every Plotly chart. No container type is unused.

DataCleaner.clean_record + clean_records()

src/cleaning.py · lines 70 – 115

```
# src/cleaning.py — conditionals + loops in one method
def clean_record(self, raw: dict) -> dict:
    city, canton = self.clean_location(raw.get("location", ""))
    features = self.clean_features(raw.get("features", []))

    # conditional — derive boolean amenity flags from a string list
    has_balcony = 1 if any("balcony" in f for f in features) else 0
    has_parking = 1 if any(
        k in f for f in features for k in ("parking", "garage")
    ) else 0
    ...

# src/cleaning.py — procedural helper with a for-loop + if-guard
def clean_records(rows: list[dict]) -> list[dict]:
    cleaner = DataCleaner()
    out: list[dict] = []
    for row in rows:                    # loop over every scraped row
        cleaned = cleaner.clean_record(row)
        if cleaned["price"] is not None and cleaned["size"] is not None:
            out.append(cleaned)         # conditional keep
    return out
```

Why We Deserve the Points

Conditionals — per-row if/else derives the balcony and parking boolean flags from a list of feature strings. Loop control — a for-loop walks every scraped row, with an inner conditional that keeps a row only when both price and size parsed successfully. data_quality.validate() layers more conditional guards on top: dedup, plausibility bounds, CHF/m² band, and IQR-based outlier removal. All five rubric ingredients are present.

class SwissHousingScraper + procedural helpers

src/scraper.py (OOP); src/database.py (procedural helper)

```
# OOP — class SwissHousingScraper  (src/scraper.py)
class SwissHousingScraper:
    """OOP scraper for flatfox.ch detail pages."""
    FACT_LABELS: dict[str, str] = {           # class-level dict
        "Number of rooms:": "rooms_raw",
        "Livingspace:":     "size",
        "Gross rent (incl. utilities):": "price",
    }
    def __init__(self, target_url=None, max_listings=30, ...):
        self.session = requests.Session()     # instance state
    def fetch(self, url: str) -> str: ...
    def parse(self, html: str) -> Listing: ...

# Procedural — clean_records() / persist_dataframe() / run_pipeline()
def persist_dataframe(df, db_path=DEFAULT_DB_PATH) -> int:
    with HousingDatabase(db_path) as db:      # context-managed OOP
        return db.save(df)                    # called from procedure
```

Why We Deserve the Points

Rubric asks for one paradigm; we use both. OOP holds the stateful collaborators: SwissHousingScraper (shared session), HousingDatabase (sqlite3 connection, context-manager protocol, 9-query catalog), LLMProcessor (OpenAI client + persistent cache). Procedural functions handle the stateless pipeline glue: clean_records(), persist_dataframe(), run_pipeline(). Each paradigm is applied where it actually fits.

report_figures (matplotlib) + plotly_* (dashboard)

src/report_figures.py + src/visualization.py · all plot fns

```
# src/report_figures.py — six report-ready PNGs (matplotlib)
def fig_size_vs_price(df, out):
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.scatter(df["size"], df["price"], alpha=0.6, s=35)
    r, _ = pearsonr(df["size"], df["price"])
    slope, intercept, *_ = linregress(df["size"], df["price"])
    ax.plot(xs, slope*xs + intercept, color="C3")
    ax.set_title(f"Size vs rent  (r = {r:.2f}, slope ≈ {slope:.0f} CHF/m²)")
    fig.savefig(out, dpi=160)

# app/streamlit_app.py — interactive Plotly inside the dashboard
fig = plotly_scatter_size_price(df_filtered)
st.plotly_chart(fig, use_container_width=True)

# Plus the printed-report 6-figure bundle:
#  01_size_vs_price.png   04_feature_impact.png
#  02_rent_by_rooms.png   05_correlation_matrix.png
#  03_rent_ranges.png     06_chf_per_m2_distribution.png
```

Why We Deserve the Points

Two visualisation families serve data exploration. Six static matplotlib report figures (size↔price, rent by rooms, rent ranges, feature impact, correlation matrix, CHF/m²) ship as PDF-ready PNGs. Ten interactive Plotly charts (scatter + OLS line, violin/box, OSM bubble map, correlation heatmap, scatter matrix, price categories) drive the Streamlit dashboard. Plus DataFrame tables in every dashboard tab — slides 11 – 17 in this deck.

statistics.correlation_price_size()

src/statistics.py · lines 121 – 149 (one of 7 tests in the battery)

```
# src/statistics.py — Pearson r WITH p-value, n, and effect size
def correlation_price_size(df: pd.DataFrame) -> TestResult:
    sub = df[["size", "price"]].dropna()
    r, p = pearsonr(sub["size"], sub["price"])     # <- p-value here
    r2 = r ** 2
    return TestResult(
        name="pearson_size_price",
        statistic=float(r),
        p_value=float(p),                          # <- carried in result
        effect_size=float(r2),
        effect_size_label=(
            f"R² = {r2:.3f} — size explains "
            f"{r2*100:.1f}% of price variance"),
        interpretation=(
            f"r = {r:.3f}.  {_p_phrase(p)}.  "
            f"Each extra m² is associated with higher rent."),
        n=len(sub),
    )
# Battery also runs: Spearman ρ, OLS, Welch t-test, Mann-Whitney U,
# Shapiro-Wilk — every one ships a p-value.
```

Why We Deserve the Points

Rubric requires a statistical analysis with a p-value. We ship seven, each returning a p-value: Pearson r (size↔price, size↔CHF/m²), Spearman ρ (rooms↔price), OLS regression with confidence interval, Welch t-test (balcony), Mann-Whitney U (parking), Shapiro-Wilk (normality check). Every TestResult also ships an effect size (R^2, Cohen's d) and a plain-language interpretation that the dashboard renders verbatim.

Price provenance + suspicious-price audit

src/collect_zurich.py · to_output(); src/data_quality.py · validate()

```
# Creative engineering #1 — never trust one price field.
# collect_zurich.to_output()  captures FIVE price fields per listing:
def to_output(rec: dict) -> dict:
    raw_text, title_price = parse_title_price(rec["public_title"])
    gross = int(rec["rent_gross"])
    mismatch = bool(title_price is not None
                    and abs(title_price - gross) >= 100)
    return {
        "rent_price":      gross,            # authoritative
        "rent_net":        rec.get("rent_net"),
        "additional_costs": rec.get("rent_charges"),
        "raw_price_text":  raw_text,         # what the user SEES
        "title_price":     title_price,      # parsed from title
        "price_mismatch":  mismatch,         # audit flag
    }

# data_quality.validate()  cross-checks title vs structured price
gap = (df["title_price"] - df["price"]).abs()
suspicious = (gap >= PRICE_MISMATCH_TOLERANCE).fillna(False)
report.n_suspicious_price = int(suspicious.sum())
report.suspicious_examples = df.loc[suspicious, AUDIT_COLS]
```

Why We Deserve the Points

Rubric: creative = what isn't in the lessons / exercises. Syllabus covered matplotlib / folium, Flask, LangChain, OOP, loops, data structures. Outside that — shown here: never-trust-one-price-field design (collector captures FIVE price fields + mismatch flag; validator cross-checks Flatfox's stale public_title bug). Also outside the curriculum: BeautifulSoup scraping, SQLite + SQL window functions, scipy.stats with effect sizes, Plotly + Streamlit (chosen over the taught folium + Flask), content-keyed LLM cache with PROMPT_VERSION.

Flatfox JSON API + BeautifulSoup scrape

src/collect_zurich.py · API; src/scraper.py · HTML detail pages

```
# src/collect_zurich.py — public JSON API (no auth) — Web API
FLATFOX_API = "https://flatfox.ch/api/v1/public-listing/"

def fetch_page(offset: int, page_size: int = 100) -> list[dict]:
    r = requests.get(FLATFOX_API,
                     params={"offset": offset, "limit": page_size,
                             "ordering": "-publication_date"},
                     headers=HEADERS, timeout=20)
    r.raise_for_status()
    return r.json().get("results", [])

# src/scraper.py — BeautifulSoup scrape of each detail page
class SwissHousingScraper:
    def parse(self, html: str) -> Listing:
        soup = BeautifulSoup(html, "html.parser")
        rooms = soup.find("dt", string="Number of rooms:")\
                    .find_next("dd").get_text(strip=True)
        return Listing(price=..., size=..., rooms=rooms, ...)
```

Why We Deserve the Points

Two network layers, both implemented in the Python code (no manual download step). Web API — the Flatfox public-listing JSON endpoint, called with requests, paginated client-side, filtered to ZH apartments. Web scraper — BeautifulSoup parses each detail page's facts list (price, size, rooms, features, description) on a 4-worker thread pool. The rubric asks for either; we deliver both, layered.

class HousingDatabase (sqlite3 + 9-query catalog)

src/database.py · lines 20 – 256 (catalog of named SQL queries)

```
# src/database.py — class HousingDatabase  (SQLite + window funcs)
class HousingDatabase:
    TABLE = "apartments"
    def __init__(self, db_path=DEFAULT_DB_PATH):
        self.conn = sqlite3.connect(db_path)
    def save(self, df: pd.DataFrame) -> int:
        df.to_sql(self.TABLE, self.conn,
                  if_exists="replace", index=False)
        self.conn.executescript("""
            CREATE VIEW v_apartments AS
            SELECT *, ROUND(price/NULLIF(size,0), 2) AS chf_per_m2
            FROM apartments;""")
    def price_distribution_by_category(self):
        return self.query("""
            WITH tiles AS (
              SELECT price, NTILE(4) OVER (ORDER BY price) AS q
              FROM apartments)
            SELECT q, COUNT(*) AS n, AVG(price) AS avg_price
            FROM tiles GROUP BY q;""")
```

Why We Deserve the Points

A real SQLite database (housing.db), populated and queried from the Python code via class HousingDatabase. The query catalog goes well beyond basic SELECT/GROUP BY: NTILE quartile bucketing, RANK-based city ranking with premium-vs-market computation, AVG(..) OVER (rolling 10-row window with frame), CTEs, UNION ALL across CASE-driven aggregates, a generated view (v_apartments). Nine named queries, all dashboard-rendered.

class LLMProcessor (OpenAI gpt-4o-mini + cache)

src/llm_helper.py · class LLMProcessor, lines 29 – 200+

```
# src/llm_helper.py — class LLMProcessor  (OpenAI gpt-4o-mini)
class LLMProcessor:
    SCHEMA_KEYS    = ("balcony", "parking", "furnished")
    PROMPT_VERSION = "v1"     # bump invalidates the SQLite cache

    def extract_features(self, description: str) -> dict:
        cache_key = self._cache_key(description)        # hash(prompt|model|desc)
        cached    = self._cache_get(cache_key)
        if cached is not None:
            self.calls_cache += 1
            return cached                                # never pay twice
        if self._client is not None:                     # OpenAI ready?
            try:
                result = self._extract_via_openai(description)
                self._cache_put(cache_key, result)
                self.calls_openai += 1
                return result
            except Exception: pass
        self.calls_fallback += 1
        return self._extract_via_regex(description)      # always-on fallback
```

Why We Deserve the Points

Rubric asks for an LLM supporting data preparation or analysis. OpenAI gpt-4o-mini reads every listing's free-text description and returns three structured boolean features (balcony, parking, furnished) the regex alone could not reliably extract. LLMProcessor is content-keyed and SQLite-cached (no duplicate API calls), versioned via PROMPT_VERSION, and falls back to regex when no API key is present.

app/streamlit_app.py (7-tab Plotly dashboard)

app/streamlit_app.py — entry-point `streamlit run app/streamlit_app.py`

```
# app/streamlit_app.py — multi-tab analytics dashboard
import streamlit as st
from src.pipeline import run_pipeline

st.set_page_config(page_title="Swiss Housing Analytics",
                   layout="wide", initial_sidebar_state="expanded")

pipe = run_pipeline()      # one entry-point = same as CLI / notebook
df   = pipe["df"]

# Sidebar filters narrow the shared DataFrame; everything re-renders.
city  = st.sidebar.multiselect("City", sorted(df["city"].unique()))
price = st.sidebar.slider("Rent (CHF)", 500, 15000, (1000, 6000))
df_f  = df[df["price"].between(*price)]

tabs = st.tabs(["Overview", "Geography", "Stats", "Insights",
                "SQL", "Listings", "Data quality"])
with tabs[0]:
    st.plotly_chart(plotly_scatter_size_price(df_f),
                    use_container_width=True)
```

Why We Deserve the Points

A multi-tab Streamlit web application (app/streamlit_app.py) presents every artefact of the pipeline in one place: cleaned data, statistical results, the SQL query catalog, the data-quality audit trail. Seven tabs (Overview, Geography, Stats, Insights, SQL, Listings, Data quality), Plotly charts with hover/zoom, and sidebar filters (city, price band, room count) that propagate through every tab and chart.